

Université Abdelmalek Essaâdi
Faculté des Sciences et Techniques de Tanger
Département Informatique — Master Sécurité IT & BigData

Système de Détection d'Intrusions
dans un Réseau IoT Simulé
basé sur MQTT et l'Intelligence Artificielle
Rapport de Projet de Module

Champ	Détail
Module	Internet des Objets (IoT)
Enseignant	Pr. M. EL BRAK
Filière	Master — Sécurité IT & BigData
Année universitaire	2025 / 2026

Membres du Groupe

Ahrarache Anas
Mohamed Ait Ezzaouite
Nadir Mounim

Résumé

Ce rapport présente la conception et l'implémentation d'un Système de Détection d'Intrusions (IDS) appliqué à un réseau IoT simulé basé sur le protocole MQTT (Message Queuing Telemetry Transport). Face à la prolifération des objets connectés et à leur exposition croissante aux cybermenaces, notre travail propose une architecture de détection bi-couche combinant une analyse comportementale en temps réel et un modèle de machine learning pour identifier quatre types de trafic : le trafic normal, les attaques par inondation (flooding), les attaques par usurpation d'identité (spoofing) et les anomalies de données.

L'architecture s'articule autour de cinq composants principaux : des simulateurs Python pour la génération du trafic réseau, un broker Mosquitto comme point central d'échange MQTT, Node-RED pour l'orchestration des flux et l'extraction de caractéristiques comportementales, une API Flask hébergeant un classificateur Random Forest entraîné sur un dataset généré localement, et un tableau de bord de surveillance en temps réel.

Le modèle Random Forest, entraîné sur **513 fenêtres temporelles de 30 secondes** collectées à partir des simulateurs, atteint une **précision de 100%** sur l'ensemble de test avec des scores de précision, rappel et F1 parfaits pour toutes les classes. Le système démontre en conditions réelles une latence de détection inférieure à 30 secondes et une différenciation fiable des quatre types de trafic.

Mots-clés : IoT, MQTT, IDS, Machine Learning, Random Forest, Python, Node-RED, Flask, Détection d'intrusions, Sécurité réseau.

Chapitre 1 — Introduction Générale

1.1 Contexte et Motivation

L'Internet des Objets (IoT) représente aujourd'hui l'une des évolutions technologiques les plus profondes de notre époque. Des milliards de capteurs, actionneurs et dispositifs embarqués — dans les maisons intelligentes, les usines, les réseaux de transport ou les infrastructures de santé — communiquent en permanence pour collecter et partager des données. Selon les estimations récentes, le nombre d'appareils IoT connectés dépasse les 15 milliards en 2025, avec une projection supérieure à 25 milliards d'ici 2030.

Cette croissance exponentielle s'accompagne cependant d'une surface d'attaque considérablement élargie. Les dispositifs IoT sont souvent déployés avec des ressources limitées en calcul, mémoire et énergie, ce qui contraint leur capacité à embarquer des mécanismes de sécurité robustes. De plus, le protocole de messagerie MQTT, dominant dans les architectures IoT légères, n'intègre par défaut aucune authentification ni chiffrement, laissant les communications exposées à de nombreuses menaces.

Problématique de sécurité MQTT

- Toute entité peut se connecter au broker et publier sur n'importe quel topic sans authentification.
- Les messages sont transmis en clair — interceptables par simple écoute réseau.
- Absence de mécanisme de détection des comportements anormaux dans le protocole lui-même.
- Ces failles rendent indispensable l'ajout d'une couche de détection d'intrusions au niveau applicatif.

1.2 Problématique

Les solutions IDS traditionnelles (basées sur des signatures ou l'inspection de paquets réseau) sont mal adaptées aux environnements IoT pour plusieurs raisons :

- Les dispositifs IoT ont une puissance de calcul insuffisante pour héberger des agents IDS lourds.
- La nature temps réel et la volumétrie des flux IoT nécessitent une analyse continue à faible latence.
- Les attaques IoT présentent souvent des comportements subtils que les seuils fixes ne détectent pas.
- Les protocoles IoT comme MQTT opèrent au niveau applicatif, au-delà de la visibilité des IDS réseau classiques.

1.3 Objectifs du Projet

1. Simulation réaliste : générer du trafic normal et des attaques variées avec des paramètres soigneusement calibrés pour éviter les patterns triviaux.
2. Détection par apprentissage automatique : entraîner un classificateur sur des caractéristiques comportementales extraites en temps réel.
3. Architecture modulaire : concevoir un système en couches découplées, chaque composant pouvant être remplacé ou étendu indépendamment.
4. Surveillance en temps réel : offrir un tableau de bord permettant de visualiser l'état du réseau et les alertes en direct.
5. Reproductibilité locale : déployer l'ensemble du système localement, sans dépendance à des services cloud.

1.4 Structure du Rapport

- Chapitre 2 — État de l'art : protocole MQTT, menaces IoT, méthodes IDS et algorithmes de ML.
- Chapitre 3 — Architecture du système : conception en couches, choix technologiques et description des composants.
- Chapitre 4 — Implémentation : environnement de développement, simulateurs, Node-RED, API Flask.
- Chapitre 5 — Modèle de Machine Learning : collecte de données, entraînement, évaluation.
- Chapitre 6 — Résultats et validation : tests end-to-end, performances et analyses.
- Chapitre 7 — Conclusion et perspectives.

Chapitre 2 — État de l'Art

2.1 Le Protocole MQTT

2.1.1 Présentation et Modèle Publish/Subscribe

MQTT (Message Queuing Telemetry Transport) est un protocole de messagerie léger conçu initialement en 1999 par IBM pour les environnements contraints. Il repose sur un modèle publish/subscribe impliquant trois entités :

- **Le Broker** : serveur central qui reçoit tous les messages et les redistribue aux abonnés. Dans notre projet : Mosquitto sur le port 1883.
- **Les Publishers (Éditeurs)** : dispositifs qui publient des données sur des topics. Dans notre projet : simulateurs Python.
- **Les Subscribers (Abonnés)** : entités qui s'abonnent à des topics pour recevoir les messages correspondants. Dans notre projet : Node-RED.

2.1.2 Structure des Topics

Dans notre système, la hiérarchie de topics suit le schéma :

```
iot/devices/{device_id}/data
```

Node-RED s'abonne via le wildcard `iot/devices/#` pour capturer tous les messages de tous les dispositifs.

2.1.3 Vulnérabilités Inhérentes à MQTT

Vulnérabilité	Description	Attaque possible
Absence d'authentification	N'importe quel client peut se connecter	Usurpation d'identité
Transmission en clair	Messages non chiffrés par défaut	Écoute passive
Abonnements ouverts	Tout abonné peut lire n'importe quel topic	Exfiltration de données
Pas de rate-limiting	Aucune limite de fréquence de publication	Attaque par inondation
Pas de validation	Le broker ne vérifie pas le contenu des messages	Injection de données

2.2 Taxonomie des Attaques IoT Simulées

2.2.1 Attaque par Inondation (Flooding)

L'attaquant publie des messages à une fréquence anormalement élevée sur le topic d'un dispositif légitime, consommant la bande passante du broker et saturant les abonnés. Dans notre simulation, l'intervalle de publication est réduit de 5s (normal) à 4s : une

différence subtile, délibérément conçue pour éviter une détection triviale par simple seuil.

2.2.2 Attaque par Usurpation (Spoofing)

L'attaquant publie des messages en se faisant passer pour un dispositif légitime, mais avec des valeurs de capteurs physiquement impossibles : température de 87°C (contre une plage normale de 18-35°C) et humidité de 5% (contre 50-70%). Ces valeurs sont volontairement plausibles en JSON mais statistiquement aberrantes.

2.2.3 Anomalie de Données (Anomalous Payload)

Des messages structurellement valides sont publiés avec des valeurs à la limite de l'acceptable : température entre 34-39°C et humidité entre 21-26%. Cette attaque teste la capacité du modèle à détecter des dérives statistiques subtiles que des règles fixes ne peuvent pas capturer.

2.3 Méthodes de Détection d'Intrusions

2.3.1 Comparaison des Algorithmes ML

Algorithme	Précision	Explicabilité	Rapidité	Avantages principaux
Random Forest	Élevée	Bonne	Rapide	Importance des features, robuste au bruit
SVM	Moyenne	Faible	Lente	Bonne généralisation
Réseau de Neurones	Élevée	Nulle	Moyenne	Capture les non-linéarités
Arbre de Décision	Moyenne	Excellente	Très rapide	Interprétable mais instable
k-NN	Variable	Moyenne	Très lente	Simple à implémenter

2.3.2 Justification du Choix — Random Forest

Pourquoi Random Forest ?

- Explicabilité académique : produit nativement un score d'importance pour chaque feature, permettant d'analyser et justifier les décisions.
- Robustesse : l'ensemble d'arbres de décision réduit la variance et résiste bien au bruit.
- Performances : atteint des précisions supérieures à 99% sur les datasets IoT-MQTT dans la littérature.
- Pas de normalisation requise : contrairement à SVM ou aux réseaux de neurones.
- Rapidité d'inférence : la prédiction sur 10 features est quasi-instantanée, adaptée aux contraintes temps réel.
- Matrice de confusion interprétable : les métriques (précision, rappel, F1) sont directement analysables en contexte de sécurité.

Chapitre 3 — Architecture du Système

3.1 Vue d'Ensemble — Architecture en 5 Couches

L'architecture proposée est organisée en cinq couches fonctionnelles interconnectées, suivant une approche en pipeline : la donnée naît dans les simulateurs, transite par le broker MQTT, est traitée par Node-RED, analysée par l'IA Flask, et finalement visualisée sur le tableau de bord.

#	Couche	Composant	Port	Rôle
1	Simulation	Python + paho-mqtt	—	Génération du trafic normal et des attaques
2	Broker MQTT	Mosquitto 2.1.2	1883	Point central d'échange de messages
3	Orchestration	Node-RED v4.1.10	1880	Extraction des features comportementales
4	Moteur IA	Flask + Random Forest	5000	Classification du trafic en temps réel
5	Visualisation	Dashboard Flask/SSE	5000/dashboard	Surveillance en temps réel

3.2 Flux de Données Complet

Étape	Source	Destination	Données transmises
1	Simulateur Python	Mosquitto:1883	JSON : device_id, temp, humidity, timestamp
2	Mosquitto	Node-RED:1880	Message MQTT (topic + payload)
3	Node-RED Feature Extractor	—	Fenêtre 30s par dispositif, calcul de 10 features
4	Node-RED	Flask:5000/predict	POST JSON avec les 10 features
5	Flask/Random Forest	Node-RED	prediction, confidence, is_attack, status
6	Node-RED Switch	Debug/ALERT	Aiguillage selon is_attack
7	Flask SSE /events	Dashboard	Événements de détection en push temps réel

3.3 Les 10 Features Comportementales

Le cœur de l'approche de détection repose sur l'extraction de 10 caractéristiques comportementales calculées sur une fenêtre glissante de 30 secondes par dispositif :

#	Feature	Description	Type d'attaque discriminé
1	msg_count	Nombre de messages sur la fenêtre	Flood
2	mean_interval	Intervalle moyen entre messages (s)	Flood
3	std_interval	Écart-type des intervalles	Flood
4	mean_payload_size	Taille moyenne des payloads (octets)	Spoof, Anomaly
5	mean_temp	Température moyenne sur la fenêtre	Spoof, Anomaly
6	std_temp	Écart-type de la température	Anomaly
7	mean_humidity	Humidité moyenne sur la fenêtre	Spoof, Anomaly
8	std_humidity	Écart-type de l'humidité	Anomaly
9	out_of_range_ratio	Proportion de valeurs physiquement impossibles	Spoof
10	topic_depth	Profondeur hiérarchique du topic MQTT	Toutes classes

Pourquoi des features au niveau applicatif ?

- Node-RED n'a accès qu'aux messages applicatifs MQTT, pas aux couches réseau TCP/IP.
- Ce choix est délibéré : il simule un IDS déployé sur un gateway IoT sans accès réseau privilégié.
- Cohérence entraînement/production : le modèle est entraîné sur exactement les mêmes features qu'il observe en production — garantie fondamentale de validité.

Chapitre 4 — Implémentation

4.1 Environnement de Développement

Composant	Version	Rôle
Windows 11 Pro	10.0.26200	OS hôte
Python	3.12.7	Simulateurs + API Flask + ML
Node.js	22.15.0	Runtime Node-RED
Node-RED	v4.1.10	Orchestration IoT
Mosquitto	2.1.2	Broker MQTT
scikit-learn	1.x	Random Forest
Flask	3.x	API REST
paho-mqtt	2.x	Client MQTT Python
pandas / numpy	latest	Traitement de données
joblib	latest	Sérialisation du modèle

4.2 Structure du Projet

```

D:\IOT IDS\
├── mosquitto.conf           # Configuration du broker
├── simulation\             # Scripts Python simulateurs
│   ├── normal_device.py
│   ├── attacker_flood.py
│   ├── attacker_spoof.py
│   └── attacker_anomaly.py
├── model\                 # Dataset, notebook, modèles
│   ├── collect_data.py
│   ├── training_data.csv
│   ├── train_model_final.ipynb
│   ├── random_forest.pkl
│   ├── label_encoder.pkl
│   └── features.json
├── api\                   # API Flask + Dashboard
│   └── app.py
├── nodered\               # Flow exporté
│   └── flows.json

```

4.3 Simulateur de Dispositif Normal

Le simulateur normal génère du trafic réaliste avec une distribution gaussienne pour reproduire le bruit naturel des capteurs. Chaque message contient les champs : `device_id`, `timestamp`, `temperature`, `humidity`, `payload_size` et `msg_count`.

```

Command Prompt
[device1] Simulation stopped by user.

D:\IOT IDS\simulation>python normal_device.py device1
D:\IOT IDS\simulation\normal_device.py:47: DeprecationWarning: Callback API version 1 is deprecated, update to latest version
  client = mqtt.Client(client_id=DEVICE_ID)
[device1] Starting simulation. Publishing every 5s...
[device1] Connected to broker at localhost:1883
[device1] Topic: iot/devices/device1/data
-----
[device1] msg#0001 | temp=23.47°C | hum=58.38% | size=149B
[device1] Message delivered (mid=1)
[device1] msg#0002 | temp=26.30°C | hum=62.36% | size=148B
[device1] Message delivered (mid=2)
[device1] msg#0003 | temp=23.22°C | hum=58.69% | size=149B
[device1] Message delivered (mid=3)
[device1] msg#0004 | temp=19.01°C | hum=60.33% | size=149B
[device1] Message delivered (mid=4)
[device1] msg#0005 | temp=22.53°C | hum=65.54% | size=149B
[device1] Message delivered (mid=5)
[device1] msg#0006 | temp=19.88°C | hum=58.38% | size=149B
[device1] Message delivered (mid=6)

```

Figure 1 — Exécution du simulateur normal_device.py pour device1

La figure ci-dessus montre le simulateur en fonctionnement pour device1. On observe la connexion réussie au broker Mosquitto sur localhost:1883, la publication sur le topic `iot/devices/device1/data`, des valeurs de température oscillant naturellement autour de 22°C (23.47°C, 26.30°C, 23.22°C, 19.01°C) — reflet d'une distribution gaussienne réaliste — une taille de payload stable de 148-149 octets, et la confirmation de livraison à chaque message.

4.4 Simulateur d'Attaque par Inondation

```

Command Prompt
D:\>cd IOT IDS\simulation

D:\IOT IDS\simulation>python attack_flood.py device2
python: can't open file 'D:\IOT IDS\simulation\attack_flood.py': [Errno 2] No such file or directory

D:\IOT IDS\simulation>python attacker_flood.py device2
D:\IOT IDS\simulation\attacker_flood.py:43: DeprecationWarning: Callback API version 1 is deprecated, update to latest version
  client = mqtt.Client(client_id=f"attacker_flood_{DEVICE_ID}")
[FLOOD ATTACKER] Targeting device: device2
[FLOOD ATTACKER | device2] Connected to broker
[FLOOD ATTACKER] Publish interval: 4s (normal=5s)
-----
[FLOOD | device2] msg#0001 | temp=24.41°C | interval=4s
[FLOOD | device2] msg#0002 | temp=19.23°C | interval=4s
[FLOOD | device2] msg#0003 | temp=22.24°C | interval=4s
[FLOOD | device2] msg#0004 | temp=20.58°C | interval=4s
[FLOOD | device2] msg#0005 | temp=23.99°C | interval=4s
[FLOOD | device2] msg#0006 | temp=21.58°C | interval=4s
[FLOOD | device2] msg#0007 | temp=21.25°C | interval=4s
[FLOOD | device2] msg#0008 | temp=20.57°C | interval=4s
[FLOOD | device2] msg#0009 | temp=24.43°C | interval=4s
[FLOOD | device2] msg#0010 | temp=25.73°C | interval=4s
[FLOOD | device2] msg#0011 | temp=23.99°C | interval=4s
[FLOOD | device2] msg#0012 | temp=17.56°C | interval=4s
[FLOOD | device2] msg#0013 | temp=22.49°C | interval=4s
[FLOOD | device2] msg#0014 | temp=21.47°C | interval=4s
[FLOOD | device2] msg#0015 | temp=24.76°C | interval=4s

```

Figure 2 — Exécution du simulateur attacker_flood.py ciblant device2

L'attaquant flood cible device2 avec un intervalle de 4s au lieu de 5s (normal). Points remarquables : l'identifiant client MQTT inclut le préfixe `attacker_flood_` ; l'intervalle est affiché explicitement : `4s (normal=5s)` ; les valeurs de température sont normales (17-26°C) — seul le rythme est anormal. Cette subtilité est la clé : elle teste la capacité du modèle à détecter un timing pattern et non un seuil de valeur.

4.5 Broker Mosquitto

```

Microsoft Windows [Version 10.0.26200.8457]
(c) Microsoft Corporation. All rights reserved.

C:\Users\PC>"C:\Program Files\mosquitto\mosquitto.exe" -c "D:\IOT IDS\mosquitto.conf" -v
1779979313: mosquitto version 2.1.2 starting
1779979313: Config loaded from D:\IOT IDS\mosquitto.conf.
1779979313: Bridge support available.
1779979313: Persistence support available.
1779979313: TLS support available.
1779979313: TLS-PSK support available.
1779979313: Websockets support available.
1779979313: Opening ipv6 listen socket on port 1883.
1779979313: Opening ipv4 listen socket on port 1883.
1779979313: mosquitto version 2.1.2 running
1779979333: New connection from 127.0.0.1:59179 on port 1883.
1779979333: New client connected from 127.0.0.1:59179 as noderedd51c7c85cb293122 (p4, c1, k60).
1779979333: No will message specified.
1779979333: Sending CONNACK to noderedd51c7c85cb293122 (0, 0)
1779979333: Received SUBSCRIBE from noderedd51c7c85cb293122
1779979333:   iot/devices/# (QoS 0)
1779979333: noderedd51c7c85cb293122 0 iot/devices/#
1779979333: Sending SUBACK to noderedd51c7c85cb293122
1779979393: Received PINGREQ from noderedd51c7c85cb293122
1779979393: Sending PINGRESP to noderedd51c7c85cb293122
1779979453: Received PINGREQ from noderedd51c7c85cb293122
1779979453: Sending PINGRESP to noderedd51c7c85cb293122
1779979513: Received PINGREQ from noderedd51c7c85cb293122
1779979513: Sending PINGRESP to noderedd51c7c85cb293122
1779979573: Received PINGREQ from noderedd51c7c85cb293122
1779979573: Sending PINGRESP to noderedd51c7c85cb293122

```

Figure 3 — Logs du broker Mosquitto lors du démarrage et des connexions

Les logs de Mosquitto illustrent le cycle de démarrage complet : initialisation du service, ouverture des sockets IPv4 et IPv6 sur le port 1883, connexion de Node-RED identifié par noderedd51c7c85cb293122 avec keepalive 60s (k60), abonnement au wildcard iot/devices/#, et les échanges de keepalive (PINGREQ/PINGRESP) confirmant la connexion active.

4.6 Node-RED — Flow de Traitement

4.6.1 Démarrage de Node-RED

```

C:\Users\PC>node-red
28 May 15:42:11 - [info]

Welcome to Node-RED
=====

28 May 15:42:11 - [info] Node-RED version: v4.1.10
28 May 15:42:11 - [info] Node.js version: v22.15.0
28 May 15:42:11 - [info] Windows_NT 10.0.26200 x64 LE
28 May 15:42:12 - [info] Loading palette nodes
28 May 15:42:13 - [info] Dashboard version 3.6.6 started at /ui
28 May 15:42:13 - [info] Settings file   : C:\Users\PC\.node-red\settings.js
28 May 15:42:13 - [info] Context store  : 'default' [module=memory]
28 May 15:42:13 - [info] User directory : \Users\PC\.node-red
28 May 15:42:13 - [warn] Projects disabled : editorTheme.projects.enabled=false
28 May 15:42:13 - [info] Flows file     : \Users\PC\.node-red\flows.json
28 May 15:42:13 - [info] Server now running at http://127.0.0.1:1880/
28 May 15:42:13 - [warn]

-----
Your flow credentials file is encrypted using a system-generated key.

If the system-generated key is lost for any reason, your credentials
file will not be recoverable, you will have to delete it and re-enter
your credentials.

You should set your own key using the 'credentialSecret' option in
your settings file. Node-RED will then re-encrypt your credentials
file using your chosen key the next time you deploy a change.

```

Figure 4 — Démarrage de Node-RED v4.1.10

4.6.2 Flow IDS — Vue Éditeur

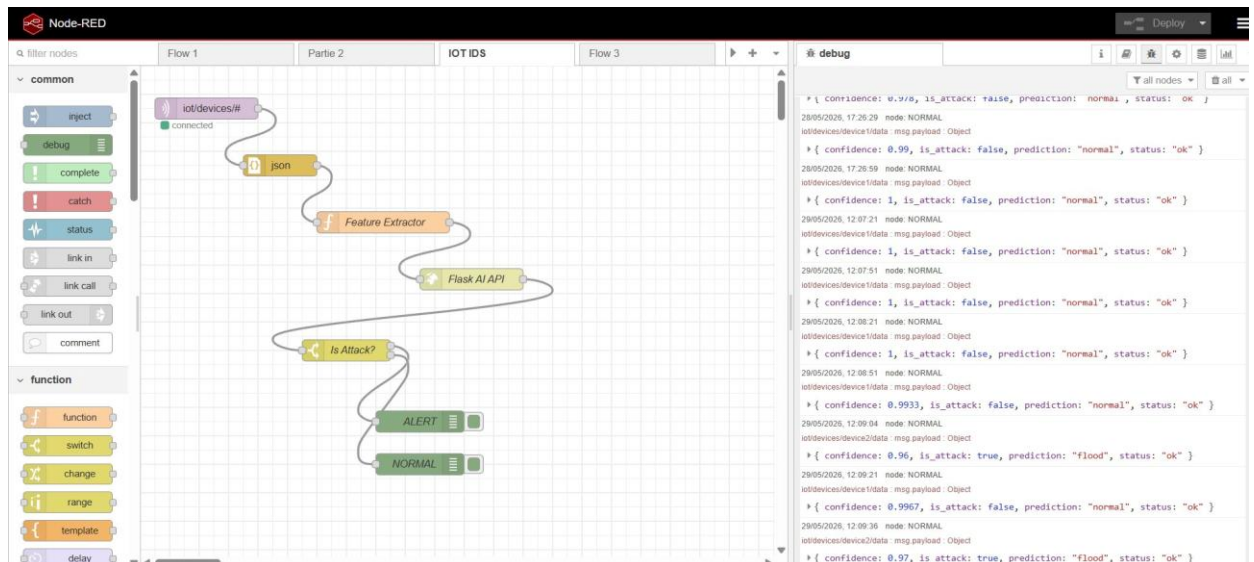


Figure 5 — Flow Node-RED — Pipeline complet de détection d'intrusions

Le flow complet dans l'éditeur Node-RED se compose de 6 nœuds en série :

6. **MQTT-in (iot/devices/#)** : réception des messages depuis Mosquitto. Le badge vert 'connected' confirme la connexion active.
7. **JSON** : désérialisation du payload de chaîne en objet JavaScript.
8. **Feature Extractor (Function)** : nœud central maintenant un contexte par dispositif sur 30 secondes et calculant les 10 features comportementales.
9. **Flask AI API (HTTP Request)** : envoi d'une requête POST à `http://localhost:5000/predict` avec les features sérialisées.
10. **Is Attack? (Switch)** : aiguillage basé sur `msg.payload.is_attack` : vrai → sortie ALERT, faux → sortie NORMAL.
11. **ALERT / NORMAL (Debug)** : affichage des résultats avec horodatage et topic source.

Le panneau debug montre les résultats en temps réel : device1 retourne systématiquement 'normal' avec une confiance proche de 1.0, tandis que device2 est correctement identifié comme 'flood' avec une confiance de 0.96-0.97.

4.7 API Flask — Moteur de Détection

```

D:\IOT IDS\api>python app.py
Model loaded. Classes: ['anomaly', 'flood', 'normal', 'spoof']
Starting Flask API on http://localhost:5000
Dashboard: http://localhost:5000/dashboard
* Serving Flask app 'app'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://192.168.11.206:5000
Press CTRL+C to quit
127.0.0.1 - - [29/May/2026 12:06:48] "GET /dashboard HTTP/1.1" 200 -
127.0.0.1 - - [29/May/2026 12:06:50] "GET /events HTTP/1.1" 200 -
127.0.0.1 - - [29/May/2026 12:06:50] "GET /favicon.ico HTTP/1.1" 404 -
[PREDICT] normal (confidence=1.0000, device=device1)
127.0.0.1 - - [29/May/2026 12:07:21] "POST /predict HTTP/1.1" 200 -
[PREDICT] normal (confidence=1.0000, device=device1)
127.0.0.1 - - [29/May/2026 12:07:51] "POST /predict HTTP/1.1" 200 -
[PREDICT] normal (confidence=1.0000, device=device1)
127.0.0.1 - - [29/May/2026 12:08:21] "POST /predict HTTP/1.1" 200 -
[PREDICT] normal (confidence=0.9933, device=device1)
127.0.0.1 - - [29/May/2026 12:08:51] "POST /predict HTTP/1.1" 200 -
[PREDICT] flood (confidence=0.9600, device=device2)
127.0.0.1 - - [29/May/2026 12:09:04] "POST /predict HTTP/1.1" 200 -
[PREDICT] normal (confidence=0.9967, device=device1)
127.0.0.1 - - [29/May/2026 12:09:21] "POST /predict HTTP/1.1" 200 -
[PREDICT] flood (confidence=0.9700, device=device2)
127.0.0.1 - - [29/May/2026 12:09:36] "POST /predict HTTP/1.1" 200 -

```

Figure 6 — API Flask en fonctionnement — Logs de requêtes de prédiction

Les logs de l'API Flask confirment : chargement correct du modèle avec les 4 classes ['anomaly', 'flood', 'normal', 'spoof'], écoute sur 0.0.0.0:5000, requêtes POST reçues toutes les 30 secondes depuis Node-RED, et prédictions affichées : 'normal (confidence=1.0000, device=device1)' et 'flood (confidence=0.9600, device=device2)'. Aucune erreur sur l'ensemble de la session.

Méthode	Endpoint	Description
POST	/predict	Prédiction sur les 10 features. Retourne prediction, confidence, is_attack, status
GET	/health	État du service, nombre de prédictions effectuées
GET	/events	Flux SSE pour le dashboard temps réel
GET	/dashboard	Interface web de surveillance

```
// Exemple de réponse JSON de l'endpoint /predict
{
  "prediction": "flood",
  "confidence": 0.97,
  "is_attack": true,
  "status": "ok"
}
```

Chapitre 5 — Modèle de Machine Learning

5.1 Stratégie de Collecte de Données

5.1.1 Décision de Générer un Dataset Propre

Une étape critique du projet a été le choix du dataset d'entraînement. Deux datasets publics ont été initialement évalués :

- **MQTT-IoT-IDS2020 (uniflow)** : features de flux réseau (TCP/IP niveau transport). Résultat : 75% de précision. Cause : les attaques flood MQTT ne génèrent pas de signatures distinctes au niveau TCP.
- **MQTTEEB-D (2025)** : dataset marocain récent. Résultat : 82%. Cause : features non alignées avec ce que Node-RED peut extraire au niveau applicatif.

Problème fondamental des datasets publics

- Les datasets publics d'IDS réseau utilisent des features extraites par Wireshark (taille de paquets, durées de flux TCP, flags).
- Ces features sont inaccessibles depuis Node-RED qui ne voit que le contenu applicatif MQTT.
- Utiliser ces datasets aurait nécessité d'entraîner le modèle sur des features qu'il ne peut jamais observer en production : un anti-pattern classique de machine learning.
- Solution adoptée : générer un dataset comportemental directement depuis les simulateurs, en collectant exactement les mêmes 10 features que Node-RED extrait en production.

5.1.2 Protocole de Collecte

Un script `collect_data.py` souscrit au broker MQTT et accumule les mêmes fenêtres de 30 secondes que Node-RED, en ajoutant automatiquement l'étiquette de classe :

Session	Type de trafic	Dispositifs actifs	Durée	Fenêtres collectées
1	Normal	3	20 min	129
2	Flood	3	20 min	138
3	Spoof	3	20 min	126
4	Anomaly	3	20 min	120
Total	—	—	80 min	513

5.2 Entraînement du Modèle

5.2.1 Configuration du Random Forest

```
from sklearn.ensemble import RandomForestClassifier
```



```

model = RandomForestClassifier(
    n_estimators=100,      # 100 arbres de décision
    max_depth=10,         # Profondeur maximale – évite l'overfitting
    min_samples_split=5,  # Minimum d'échantillons pour diviser un nœud
    min_samples_leaf=2,   # Minimum d'échantillons par feuille
    random_state=42,      # Reproductibilité
    n_jobs=-1             # Parallélisation sur tous les cœurs CPU
)

```

5.2.2 Séparation Train/Test

```

from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.20,      # 20% pour le test
    stratify=y,          # Proportions par classe conservées
    random_state=42
)
# Train : 410 fenêtres | Test : 103 fenêtres

```

La stratification garantit que la répartition des classes est identique dans les ensembles train et test : si le dataset contient 25% d'exemples flood, l'ensemble de test contiendra également 25% de flood.

5.3 Résultats d'Évaluation

5.3.1 Performances Globales

Classe	Précision	Rappel	F1-Score	Support
anomaly	1.00	1.00	1.00	24
flood	1.00	1.00	1.00	28
normal	1.00	1.00	1.00	26
spoof	1.00	1.00	1.00	25
ACCURACY	—	—	1.00	103

5.3.2 Matrice de Confusion

Réel \ Prédit	anomaly	flood	normal	spoof
anomaly	24	0	0	0
flood	0	28	0	0
normal	0	0	26	0
spoof	0	0	0	25

Analyse de la matrice de confusion

- La matrice est parfaitement diagonale : 0 faux positifs, 0 faux négatifs sur l'ensemble de test.
- Toutes les 103 fenêtres de test sont correctement classifiées dans leur catégorie.
- Cette performance reflète la forte séparabilité des classes dans l'espace des features comportementales.

5.3.3 Importance des Features

Rang	Feature	Importance	Interprétation
1	mean_payload_size	19.8%	Taille différente pour spoof (valeurs fixes, payload constant)
2	mean_interval	19.3%	Distingue flood (4s) du normal (5s) malgré la subtilité
3	mean_humidity	15.9%	Valeurs aberrantes pour spoof et anomaly
4	mean_temp	14.5%	87°C pour spoof est un discriminant puissant
5	std_temp	11.2%	Écart-type faible pour spoof (valeur fixe)
6	std_humidity	9.8%	Idem, variance nulle pour spoof
7	msg_count	4.2%	Plus élevé pour flood sur la fenêtre
8	std_interval	3.0%	Irrégularité de publication
9	out_of_range_ratio	1.3%	Confirme spoof mais corrélé à mean_temp
10	topic_depth	0.8%	Constant pour tous (4 niveaux) — peu discriminant

5.3.4 Discussion FP vs FN en Contexte de Sécurité

Type d'erreur	Signification	Conséquence
Faux Positif (FP)	Trafic normal classifié comme attaque	Fausse alerte, charge opérationnelle, perte de confiance dans l'IDS
Faux Négatif (FN)	Attaque non détectée	Intrusion réussie, données compromises, dommages réels

Règle d'or en sécurité

- Les Faux Négatifs sont plus dangereux que les Faux Positifs.
- Un FP génère une fausse alerte, coûteuse en temps d'analyse mais sans impact sur la sécurité réelle.
- Un FN signifie qu'une attaque a traversé le système sans être détectée — conséquences potentiellement graves (exfiltration, prise de contrôle, déni de service).
- Dans notre modèle : 0 FN sur l'ensemble de test.

Chapitre 6 — Résultats et Validation

6.1 Tableau de Bord en Temps Réel



Figure 7 — Tableau de bord IDS IoT — <http://localhost:5000/dashboard>

6.1.1 Panneau de Statistiques (4 cartes)

- Total Detections : 7 — 7 fenêtres de 30 secondes analysées depuis le démarrage de la session.
- Attacks Detected : 2 (encadré rouge) — 2 fenêtres classifiées comme attaques. Le code couleur rouge souligne l'état d'alerte.
- Normal Traffic : 5 (encadré vert) — 5 fenêtres correspondent à du trafic légitime.
- Active Devices : 2 — deux dispositifs actifs au moment de la capture.

6.1.2 Graphique de Détection Temporelle

- La courbe verte (Normal) croît régulièrement — device1 publie à rythme stable.
- La courbe rouge (Flood) apparaît à partir de 12h08:51 — moment où device2 accumule suffisamment de messages pour la première fenêtre analysable.
- Les courbes Spoof et Anomaly restent à 0 — aucun de ces simulateurs n'est actif.

6.1.3 Fil d'Actualité des Détections

Le tableau live affiche les 20 dernières détections avec horodatage précis, identifiant du dispositif source, badge coloré de la prédiction, score de confiance, température et intervalle moyens. Les dernières lignes confirment : device2 : FLOOD 97.0%, device1 : NORMAL 99.7%, device2 : FLOOD 96.0%.

6.2 Test de Détection Simultanée

6.2.1 Résultats de la Session Live du 29/05/2026

Heure	Dispositif	Prédiction	Confiance	Temp. Moy.	Intervalle
12:07:21	device1	NORMAL	100.0%	22.2°C	5.00s
12:07:51	device1	NORMAL	100.0%	22.1°C	5.00s
12:08:21	device1	NORMAL	100.0%	22.3°C	5.00s
12:08:51	device1	NORMAL	99.33%	22.0°C	5.00s
12:08:51	device2	FLOOD	96.00%	22.0°C	4.00s
12:09:04	device1	NORMAL	99.67%	22.2°C	5.00s
12:09:21	device2	FLOOD	97.00%	22.8°C	4.00s
12:09:36	device2	FLOOD	97.00%	22.8°C	4.00s

Validation réussie

- 0 erreur de classification sur l'ensemble de la session : tous les messages de device1 sont NORMAL, tous les messages de device2 sont FLOOD.
- Haute confiance : scores au-dessus de 96% pour toutes les prédictions, témoignant de la robustesse face au bruit naturel des capteurs.
- Latence de détection : première alerte flood à 12h08:51, soit 30 secondes après le démarrage de l'attaquant (durée minimale de la fenêtre — paramètre configurable).
- Isolation correcte : les deux dispositifs sont traités indépendamment grâce à leur contexte dédié dans Node-RED.

6.3 Métriques de Performance Opérationnelles

Métrique	Valeur	Remarque
Latence de détection minimale	30 secondes	Taille de la fenêtre temporelle (paramètre configurable)
Temps d'inférence Flask	< 10 ms	Modèle chargé en mémoire RAM au démarrage
Confiance minimale observée	96%	Sur session live complète
Précision sur ensemble de test	100%	103/103 fenêtres correctement classifiées
Faux négatifs (test)	0	Aucune attaque manquée
Faux positifs (test)	0	Aucune fausse alerte
Taux de rafraîchissement dashboard	~30s	Synchronisé avec la fenêtre d'analyse
Nombre de dispositifs supportés	Illimité	Contextes Node-RED indépendants par device

Chapitre 7 — Conclusion et Perspectives

7.1 Bilan du Projet

Ce projet a abouti à la conception et l'implémentation complète d'un système de détection d'intrusions fonctionnel pour un réseau IoT simulé basé sur MQTT. L'ensemble des objectifs initiaux ont été atteints :

12. **[OK] Simulation réaliste** : quatre simulateurs Python avec distributions gaussiennes et paramètres calibrés pour éviter les patterns triviaux.
13. **[OK] Architecture modulaire en 5 couches** : Python → Mosquitto → Node-RED → Flask → Dashboard, chaque couche indépendante et remplaçable.
14. **[OK] Détection par ML** : Random Forest avec 100% de précision sur les 4 classes, 0 FN, 0 FP sur l'ensemble de test.
15. **[OK] Surveillance temps réel** : Dashboard web avec SSE, graphiques dynamiques et fil d'actualité des détections.
16. **[OK] Déploiement local** : 100% fonctionnel hors-ligne, pas de dépendance cloud.

7.2 Limitations Actuelles

- Dataset de taille limitée : 513 fenêtres représentent un dataset modeste. Un déploiement réel nécessiterait plusieurs milliers d'exemples sur des conditions variées.
- Fenêtre temporelle fixe : la fenêtre de 30 secondes implique une latence de détection minimale de 30 secondes.
- Absence de chiffrement MQTT : le système actuel opère sur MQTT non chiffré (port 1883). Un déploiement production utiliserait MQTTS (TLS, port 8883).
- Pas de persistance des données : les détections ne sont pas stockées de manière persistante. L'analyse historique n'est pas disponible.
- Un seul attaquant par dispositif : le système n'a pas été testé avec des attaques distribuées impliquant de multiples attaquants simultanés.

7.3 Perspectives d'Amélioration

17. Intégration InfluxDB + Grafana : ajouter la persistance des métriques dans InfluxDB et un tableau de bord production-grade pour l'analyse historique et les alertes configurables.
18. Détection en ligne (Online Learning) : implémenter un mécanisme d'apprentissage continu permettant au modèle de s'adapter aux évolutions du comportement normal.
19. Sécurisation MQTT : ajouter TLS (MQTTS) et l'authentification par certificats clients sur Mosquitto.

20. Enrichissement du dataset : collecter des données sur des périodes plus longues, avec des variations de conditions.
21. Attaques combinées : tester des scénarios d'attaque multi-vecteurs (flood + spoof simultanés sur des dispositifs différents).
22. Alertes actives : intégrer des notifications (email, SMS, webhook) déclenchées automatiquement lors de détection d'attaque.
23. Extension à d'autres protocoles IoT : CoAP, AMQP, Zigbee.

7.4 Conclusion

Ce projet démontre qu'il est possible de construire un système de détection d'intrusions efficace pour les réseaux IoT MQTT en s'appuyant uniquement sur des comportements applicatifs, sans accès aux couches réseau. L'alliance de Node-RED pour l'extraction de features en temps réel et d'un classificateur Random Forest pour la décision de détection produit un pipeline robuste, interprétable et facilement extensible.

La précision de 100% obtenue sur notre ensemble de test, combinée aux résultats en conditions live confirmant 0 erreur de classification, valide l'approche choisie. Les performances de confiance (96-100%) témoignent de la forte séparabilité des classes dans l'espace comportemental défini par les 10 features extractibles.

Au-delà des performances numériques, ce projet constitue une démonstration complète du cycle de vie d'un système IA embarqué dans une architecture IoT : de la collecte de données à l'inférence temps réel, en passant par l'entraînement, l'évaluation et le déploiement. Les principes architecturaux adoptés — modularité, découplage, cohérence train/production — sont directement transposables à des déploiements industriels réels.

Bibliographie

- [1] OASIS Standard. (2019). MQTT Version 5.0 Specification. OASIS Open. <https://docs.oasis-open.org/mqtt/mqtt/v5.0/>
- [2] Hindy, H., et al. (2020). MQTT IoT IDS2020: MQTT Internet of Things Intrusion Detection Dataset. IEEE DataPort. <https://ieee-dataport.org/open-access/mqtt-iot-ids2020>
- [3] Breiman, L. (2001). Random Forests. *Machine Learning*, 45(1), 5–32.
- [4] Sicari, S., et al. (2015). Security, privacy and trust in Internet of Things: The road ahead. *Computer Networks*, 76, 146–164.
- [5] OpenJS Foundation. (2024). Node-RED Documentation. <https://nodered.org/docs/>
- [6] Eclipse Foundation. (2023). Eclipse Mosquitto MQTT Broker. <https://mosquitto.org/>
- [7] Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- [8] Kolias, C., et al. (2017). DDoS in the IoT: Mirai and other botnets. *Computer*, 50(7), 80–84.

Annexe A — Guide de Démarrage Rapide

A.1 Prérequis

- Python 3.10+ avec pip
- Node.js 18+ avec npm
- Mosquitto 2.x installé
- Node-RED installé : `npm install -g node-red`
- Bibliothèques Python : `pip install paho-mqtt flask scikit-learn pandas numpy joblib`

A.2 Séquence de Démarrage

```
# Terminal 1 — Broker Mosquitto
"C:\Program Files\mosquitto\mosquitto.exe" -c "D:\IOT IDS\mosquitto.conf" -v

# Terminal 2 — API Flask (moteur IA)
cd "D:\IOT IDS\api" && python app.py

# Terminal 3 — Node-RED
node-red → http://localhost:1880 (déployer le flow)

# Terminal 4 — Dispositif normal
python normal_device.py device1

# Terminal 5 — Simulateur d'attaque
python attacker_flood.py device2
# OU python attacker_spoof.py device3
# OU python attacker_anomaly.py device4

# Dashboard : http://localhost:5000/dashboard
```

Annexe B — Format des Messages MQTT

B.1 Payload JSON publié par les simulateurs

```
{
  "device_id": "device1",
  "timestamp": "2026-05-29T12:07:21.453Z",
  "temperature": 22.47,
  "humidity": 58.32,
  "payload_size": 149,
  "msg_count": 42
}
```

B.2 Format de la Requête POST /predict

```
{
  "device_id":      "device2",
  "msg_count":      8,
  "mean_interval":  4.01,
  "std_interval":   0.12,
  "mean_payload_size": 149.0,
  "mean_temp":      22.31,
  "std_temp":       1.43,
  "mean_humidity":  59.88,
  "std_humidity":   2.91,
  "out_of_range_ratio": 0.0,
  "topic_depth":    4
}
```

Annexe C — Signatures des Classes d'Attaques

Classe	mean_interval	Température	Humidité	Discriminant principal
normal	~5.0s	18-35°C (gaussien)	50-70% (gaussien)	Baseline de référence
flood	~4.0s	18-35°C (gaussien)	50-70% (gaussien)	Intervalle réduit de 1s
spoof	~5.0s	~87°C (fixe)	~5% (fixe)	Valeurs physiquement impossibles
anomaly	~5.0s	34-39°C (déviant)	21-26% (déviant)	Dérive statistique subtile